AutoBayes brms-parity loop: Tier C review

Status at the Tier C boundary, prepared for Andrew

11 October 2026

This is the Tier C boundary review, in the same form as the Tier A and B ones. All seven Tier C features, and the two-reference acceptance you approved at the Tier B boundary (P6), are implemented and pass every gate. They have not all reached `main`: since 11 October about 21:30 UTC, GitHub Actions has refused to start any job on your account ("recent account payments have failed or your spending limit needs to be increased"), so the merge queue, which waits for hosted CI, stopped after P6 and C05. The other six features are verified locally as one chain on top of `main` and are ready to merge in order once CI runs again. Section 6 is the part that needs you.

1 What was built

Table 1 lists the features.
Spline phase 2 (`s()`) is not in this list: it waits for your licensing decision (Section 6). Every agent wrote its feature from brms's documentation, Stan data and outputs, never from brms's, loo's or mgcv's GPL source; C07's PSIS follows ArviZ's Apache-2.0 implementation and the published papers, with attribution in the docstrings.

How close. Measured with the gates' own comparisons on the chain's final tree: all 97 ungated cases pass, with worst relative errors over every case of 8.4×10^{-10} in the log density, 7.2×10^{-9} in the gradient, 5.2×10^{-14} in predictions and 4.2×10^{-15} between the JAX and PyTensor densities. The 25 cases new in Tier C and P6 sit near 10^{-15}, with three exceptions:

- the log density of mixtures under brms's default half-Student-t priors (up to 5.8×10^{-10}), which is PyMC's float32 $\log 2$, the follow-up from Tier B;
- the cloglog cases, where brms's own arithmetic is inexact: its log density by up to 1.9×10^{-10}, and its gradient by 6.3×10^{-9} on `binomial_cloglog`, all inside tolerance;
- the exact Gaussian process, whose Cholesky factor of a matrix with condition number up to 10^7 costs 8.9×10^{-12} in the gradient and 5.2×10^{-14} in predictions.

The three cases accepted against two references are reported in #47: AutoBayes is within 3.7×10^{-14} of the 50-digit reference wherever brms is not.

2 What combining the features found

Because CI was down, the six unmerged features were rebased one onto the next and each was gated, and the whole non-sampling suite run, on the combined tree. Every link passed: 2,255 tests after C01, 2,355 after C03, 2,498 after C02, 2,609 after C04 and 2,748 after C06, with no failure at any step. Combining them was worth doing, because each feature had been written against a `main` that lacked the others, and several pairs silently accepted something no brms case verifies:

Feature	State	What a user can now write	Frozen brms cases it passes	Diverg.
P6 two references	merged #47	`link="cloglog"`, `cens()`, `trunc()`: accepted where brms is exact and against a committed 50-digit reference where brms is not	`bernoulli_cloglog`, `gauss_censored`, `gauss_truncated`, new `binomial_cloglog`	D18, D24, D25
C05 mixtures	merged #48	`ab.mixture(f, data, Priors(components=[...], theta=ab.Dirichlet(...)), families=[...])`: Gaussian components, ordered intercepts	`mixture_gauss_gauss` and two new	D52–D55
C07 post-processing	PR #49, CI blocked	`ab.log_lik`, `ab.bayes_R2`, `ab.loo` (PSIS-LOO with loo 2.9.0's tables)	three frozen and three new `*_post` cases	D48–D51
C01 multivariate	local, gated	`ab.gaussian("mvbind(y, y2) ~ ...", ..., rescor=True)`, `Priors(resp={...}, rescor=ab.LKJ(eta))`	`gauss_mv_rescor`, `_norescor`, new `_slopes`	D56–D59
C03 Gaussian processes	local, gated	`gp(x)` (exact) and `gp(x, k = 10, c = 5/4)` (Hilbert-space), `Priors(sdgp=..., lscale=...)`	`gauss_gp_exact`, `_approx`	D64–D67
C02 non-linear	local, gated	`ab.gaussian(["y ~ a * exp(b * x)", "a ~ 1 + (1 | g)", "b ~ 1"], ..., nl=True)`, `Priors(nlpars={...})`	`gauss_nonlinear_exp`, new `_terms`	D60–D63
C04 autocorrelation	local, gated	`ar(time, gr, p)`, `ma()`, `arma()` (regression form), `Priors(ar=..., ma=...)`	`gauss_ar1`, `gauss_arma11`, new `_arma22_varint`	D68–D71
C06 `mi()`, `me()`	local, gated	`["y ~ mi(xm) + w", "xm | mi() ~ w"]`; `me(xobs, sdx)` with `Priors(meanme=..., sdme=...)`	`gauss_mi`, `gauss_me`	D72–D75

Table 1: The Tier C features and P6: where each stands, what it offers, and the frozen brms cases it passes.

- `gp()` inside a multivariate formula built a model (a process per response); `gp()` next to `ar()` was built as well, and a third formula with `gp()` in an `mi()` model would have put a process inside it.
- `ar`/`ma` priors at the top level of a multivariate, mixture or non-linear model, and `sdgp`/`lscale` on a multivariate model or a mixture, were accepted and ignored.
- In a non-linear model, `me()` in a parameter's formula was accepted once priors were given, and `mi()` there crashed with an index error.

All of these are now refused with a message naming the construct, each with a test. The rule applied is the loop's: a combination is offered only where a frozen brms case verifies it. Several of these refusals are stricter than brms, which accepts, for example, `gp()` next to `ar()` and in multivariate formulas; they are listed in the divergence rows.

A real Linux failure. C07's first hosted run, before the billing block, failed 12 of 2,133 tests on Linux: its loo probes regenerated their random inputs, and numpy's normal and Student-t draws go through `exp`/`log`, whose last bit differs between macOS and Linux builds, so an exact hash of the inputs differed. The probes now check the inputs rounded to 8 decimals, and I confirmed that all 30 still match loo's outputs with a third of every input moved by one ulp. The fix has not yet run on Linux.

3 Where AutoBayes knowingly differs from brms (Tier C rows)

	area	what differs	status
D18, D24, D25	cloglog, `cens()`, `trunc()`	brms inexact at some frozen points; accepted against the 50-digit reference there	accepted (P6)
D48	post-processing options	`newdata`, `re_formula`, moment matching, `reloo`, `loo_compare` not offered	kept
D49	`log_lik` at extremes	brms returns ±Inf or NaN at extreme linear predictors; ours finite	documented
D50	loo MCSE	loo's is NaN below $\text{elpd}_i \approx -372$; ours finite	documented
D51	loo relative efficiency	differs for chains of ten draws or fewer	documented
D52–D55	mixtures	other component families, `order = "none"`, per-component formulas, group terms, offsets, `gp()`, `ar()`	kept / parked
D56–D59	multivariate	per-response formulas (except with `mi()`), `|ID|`, other families, `gp()`, `ar()`; names	kept
D60–D63	non-linear	nested parameters, `nlf()`, other families, `gp()`/`mo()`/`ar()`/`mi()`/`me()`, `loop = FALSE`	kept
D64–D67	Gaussian processes	other kernels, `by`, several covariates; exact GP at new inputs; brms's default-prior solver stops short of a rounding boundary on 2 of 309 probes	kept / documented
D68–D71	autocorrelation	`cov = TRUE`, `cosy`/`car`/`sar`/`fcor`, other contexts, new-data predictions	kept
D72–D75	`mi()`, `me()`	other contexts and families, `me()` on new rows, post-processing for these models	kept

4 How the claims were checked, and their limits

- Each feature: the gate tool on the tree that will merge; the full non-sampling suite on the combined tree; the adapter and gate changes read for weakening (all extensions; C06 made two response checks stricter, since a NaN in a response had compared as equal); every earlier case model unchanged.
- Reproducibility: `oracle/regen.py check -all` regenerated all 98 frozen fixtures from R on the chain's final tree and compared them with the committed files: all 98 identical.

Limits I know of. The two new fitting paths (multivariate and non-linear models are built without Bambi) are tested only with the sampler replaced, because nothing in the loop samples; the first real fit of either is untested. C06 accepts a few combinations beyond its frozen cases, checked against hand-written `scipy` densities rather than brms: several `mi()` terms, imputations that read other imputed columns, and `me()` beside other terms. `binomial_cloglog`'s seeded points were drawn from a narrower box after brms's own density was $-\infty$ at the default one. The hosted sampling tests have not run on anything after C05.

5 Process and cost

Agent tokens reported: P6 0.63M, C05 0.80M, C07 0.68M, C01 0.75M, C03 0.75M, C02 0.91M, C04 0.70M, C06 0.88M, plus 1.59M for the three rebases handed back to their agents (C02 0.31M, C04 0.92M, C06 0.36M), about 7.7M in all, against my estimate of 4.5–7M. Resuming an agent for its rebase was the expensive step: a resumed agent re-reads its whole earlier context. I would do rebases myself in future, as I did for C07, C01 and C03. The machine's load reached 141 while three agents ran their suites at once; after that I ran nothing in parallel.

6 Decisions

1. **CI.** Either restore GitHub Actions (billing or spending limit), after which I merge #49 and the five local features one at a time, each with hosted CI (about half an hour each); or tell me to merge on local verification alone, which means the full non-sampling suite and the oracle on each tree, without the sampling tests hosted CI runs.
2. **Splines.** AutoBayes is MIT; the spline module from phase 1 follows mgcv's (GPL) R functions step by step. My recommendation is a clean-room rewrite from Wood's published algorithms, keeping the same tests against mgcv's output; the alternatives are licensing the module or the package under the GPL, or dropping `s()`.
3. **Attribution.** C07's PSIS code follows ArviZ's (Apache-2.0) steps, with attribution in the docstrings. Do you want a formal `NOTICE` file as well?
4. **Cleanup.** Twenty-seven local worktrees now belong to merged or closed branches: the twenty-four listed in the Tier B review, plus `docs-tierB`, `parity-p6` and `parity-c05`. The six unmerged Tier C branches, `parity-b04` and the three permanent checkouts stay. I will remove the twenty-seven only on your yes.

Follow-ups found and not done. PyMC's half-Cauchy and half-Student-t priors add $\log 2$ in float32 (1.9×10^{-9} per prior); mixtures have no per-row log density yet, so `log_lik()` and `loo()` are not offered for them; `weights()` for the zero-inflated and hurdle families; the learned-ν `describe()`

order that depends on Python's hash seed; frozen brms cases for the combinations refused above, wherever you want them.